



SAPIENZA
UNIVERSITÀ DI ROMA

Emulatore CUDA

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Laurea Triennale in Ingegneria Informatica

Gabriele Guido

Matricola 2144286

Relatore

Prof. Giorgio Grisetti

Anno Accademico 2023/2024

Emulatore CUDA

Tesi di Laurea. Sapienza Università di Roma

© 2026 Gabriele Guido. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: guido.2144286@studenti.uniroma1.it

Sommario

La programmazione in CUDA (*Compute Unified Device Architecture*) è il modo più efficace per scrivere programmi eseguibili su GPU (*Graphics Processing Unit*). L'architettura delle schede grafiche consente numerose operazioni parallele, riducendo i tempi di esecuzione e aumentando il **throughput** (attività completate nell'unità di tempo). Negli ultimi anni l'utilizzo di questa tecnologia si è diffuso ulteriormente grazie alla velocità di esecuzione di calcoli sulle matrici e programmare in questa modalità è diventata quindi una competenza richiesta. Il suo punto di forza tuttavia è ciò che rende la progettazione degli algoritmi meno semplice: i dati sono elaborati in parallelo e senza ordine perciò durante l'esecuzione potrebbero verificarsi delle dipendenze dati che, se non trattate correttamente, porterebbero a risultati sbagliati. In CUDA abbandoniamo il concetto di esecuzione sequenziale: in ogni istante un numero finito di thread esegue lo stesso codice su una porzione dell'input dato al programma. Senza un ordine tra questi diventa necessario disegnare algoritmi in cui i thread lavorino in modo indipendente tra loro.

Il progetto nasce dal bisogno di programmare in CUDA anche senza una scheda grafica sul proprio PC e consiste in una libreria in cui ci sono le funzionalità essenziali per testare la correttezza di un algoritmo destinato al calcolo parallelo su GPU. A livello prestazionale non c'è alcun vantaggio, la vera utilità sta nella scrittura e verifica del funzionamento di algoritmi destinati al calcolo parallelo.

Indice

1	CUDA	1
2	MyCuda	3
2.1	Parametri	3
2.2	Strumenti principali	3

CUDA

Per capire come funziona l'esecuzione di un programma su una GPU dobbiamo abbandonare la concezione sequenziale e iniziare a pensare a cosa succede nello spazio più che nel tempo. Prenderemo come esempio di input da processare dei buffer di N numeri (interi). Supponiamo di voler eseguire una moltiplicazione per uno scalare S in ogni cella di $dest[N]$ e salvare il risultato su $src[N]$. Su una CPU non serve altro che iterare l'operazione su tutte le N posizioni del nostro vettore.

```
1  for(int i = 0; i < N; i++)
2      dest[i] += S * src[i];
```

Volendo ottenere lo stesso risultato su una GPU, iterare in questo modo non avrebbe senso: per sfruttare la potenza di calcolo dobbiamo ottimizzare l'esecuzione. In questo esempio dobbiamo pensare che le operazioni di moltiplicazione stavolta vengono fatte su tutte le celle contemporaneamente: ogni thread esegue lo stesso codice su una singola posizione dell'array.

```
1      dest[idx] += S * src[idx];
```

In questo modo il costo di questa semplice operazione passa da essere lineare ($O(n)$) a costante ($O(1)$).

Griglia, Blocchi e Thread Per poter gestire input con dimensione maggiore dei core (e quindi thread) che abbiamo a disposizione, l'esecuzione viene spezzata in blocchi: ogni blocco è grande quanto il numero massimo di thread a disposizione. L'input viene processato come una griglia di blocchi quindi CUDA mette a disposizione dei programmatori le informazioni come numero di thread (in un blocco), numero di blocco (nella griglia) e dimensione del blocco. In questo modo possiamo tenere conto dello spostamento spaziale della nostra esecuzione, processando il nostro input come se fosse composto da tante "mattonelle" (da non confondere con le celle di un array).

MyCuda

La libreria MyCuda contiene delle funzionalità utili alla progettazione e simulazione del calcolo parallelo. L'esecuzione effettiva è gestita tramite un numero limitato di thread che simulano i core di una GPU. Il funzionamento generale è gestito tramite una struttura dati *MyCudaItem* al cui interno troviamo i buffer di memoria (con rispettiva dimensione) e il puntatore alla funzione scritta in "*pseudo-CUDA*" che verrà eseguita simulando i core della GPU. Nella libreria la maggior parte delle funzioni sono delle operazioni ausiliare che facilitano la fase di sviluppo e verifica: gestione dei 2 buffer (creazione, stampa ed eliminazione), il calcolo di valori speciali (dimensione griglia) e verifica di condizioni necessarie (dimensione buffer). Il cuore della libreria consiste nella funzione che simula la chiamata al *Kernel* della GPU e tutta la gestione, tramite strutture dati dedicate, di indici, parametri e argomenti usati dai thread. Al di là quindi delle varie funzionalità operative, l'essenza di questa libreria è la possibilità di scrivere funzioni (in C) similmente a come si farebbe usando CUDA.

Nota: L'intera libreria è stata pensata per trattare array di interi ma la meccanica può essere facilmente estesa ad altri tipi di dato.

2.1 Parametri

Nel header *myCuda.h* troviamo le **macro** usate nella libreria. L'unica veramente rilevante è `BLOCK_DIM`, che indica la dimensione di ogni blocco e quindi il numero massimo di thread generati. Le altre sono destinate a debugging e test.

2.2 Strumenti principali

MyCudaItem Ho scelto di usare una struttura dati per salvare gli elementi più ricorrenti e rendere tutte le funzioni consistenti con questo tipo di dato. Le funzioni di libreria avranno sempre, come parametro, un puntatore a un oggetto di questo tipo, lanciando un errore in caso non sia rispettata questa sintassi.

```

1 struct MyCudaItem{
2     int  buffer_size;
3     int  size;
4     int* src_buffer;
5     int* dest_buffer;
6     KernelLaunchFn kernel_fn;

```

```
7 };
```

MyCudaKernelLaunch In questa funzione vengono creati i thread che andranno a eseguire la funzione puntata dal campo di un oggetto MyCudaItem. Per rispettare la struttura a griglia dell'esecuzione, i thread sono generati ad ogni iterazione di un singolo blocco, passandogli come funzione da eseguire proprio la funzione *kernel_fn*. Per gestire il passaggio dei parametri a ogni thread ho creato una struttura dati apposita,

```
1 struct ThreadArgs{
2     int threadIdx;
3     int blockIdx;
4     void* dest;
5     void* src;
6     int size;
7 };
```

inizializzata con MyCudaSetThreadArgs(). In questo modo ogni thread potrà accedere alla sua posizione e ai buffer. Con MyCudaIterations() vengono calcolati i passaggi totali per consumare l'input, ovvero il numero di blocchi totali.

```
1 void MyCudaKernelLaunch(MyCudaItem* cuda_item){
2     assert(cuda_item && "cuda_item not valid");
3     int passes = MyCudaIterations(cuda_item);
4     pthread_t threads[BLOCK_DIM];
5     ThreadArgs thread_arguments[BLOCK_DIM];
6     for(int bid = 0; bid < passes; bid++){
7         for(int tid = 0; tid < BLOCK_DIM; tid++){
8             MyCudaSetThreadArgs(&thread_arguments[tid], tid
9                                 ,bid,cuda_item->dest_buffer,cuda_item->
10                                 src_buffer,cuda_item->size);
11             //kernel launch:
12             pthread_create(&threads[tid],NULL,cuda_item->
13                           kernel_fn,&thread_arguments[tid]);
14         }
15         pthread_join(threads[tid],NULL);
16     }
17 }
```

Una volta entrati nella funzione, i parametri vengono estratti dalla `struct` e salvati localmente. Per vedere il loro utilizzo prendiamo l'esempio di `ReductionSum`, in cui sommiamo tutti i valori di un vettore. Nella sua versione `Kernel` troviamo

```
1 void* MyCudaReductionKernel(void* args){
2     ThreadArgs* info = (ThreadArgs*) args;
3
4     //estraggo i parametri
5     int idx = info->threadIdx+info->blockIdx*BLOCK_DIM;
6     int size = info->size;
7     int* src = info->src;
8     int* dest = info->dest;
9
10    //verifico se sono ancora nel buffer
11    if(idx >= size){
12        pthread_exit(NULL);
13    }
14
15    //sommo due celle vicine
16    dest[idx] = src[2*idx]+src[2*idx+1];
17    pthread_exit(NULL);
18 }
```

MyCudaMemSwap In molti algoritmi, prima di arrivare al risultato, ci sono diversi passaggi intermedi in cui si salvano risultati da usare successivamente. Per evitare di consumare molto spazio usiamo i due array di sorgente e destinazione in modo alternato. Questa funzione non fa altro che invertire, all'interno della struttura `MyCudaItem`, i puntatori alle due zone di memoria. Ad esempio nel caso di `ReductionSum` gli array vengono alternati finché la dimensione finale è 1 (la somma totale).

```
1 void MyCudaMemSwap(MyCudaItem* cuda_item){
2     assert(cuda_item && "CudaItem_not_valid");
3     int* aux = cuda_item->src_buffer;
4     cuda_item->src_buffer = cuda_item->dest_buffer;
5     cuda_item->dest_buffer = aux;
6     for(int i = 0; i<cuda_item->buffer_size;i++){
7         cuda_item->dest_buffer[i] = 0;
8     }
9 }
10 }
```

